Skill `scraper`

Web scraping ético y robusto para Claude Code

De una guía "solo instrucciones" a una skill **determinista, auditable y reutilizable**

El problema

Una skill "solo instrucciones" describe buenas prácticas, pero **Claude decide cada vez** qué librerías usa y cómo:

- ¿Pone retry/backoff? ¿Respeta robots.txt? ¿Guarda incrementalmente?
- No hay garantía → comportamiento **impredecible** y código distinto cada sesión.

Pregunta clave: ¿cómo sabemos qué herramientas va a usar?

La solución: código fijo + límites declarados

Qué se usa	Cómo queda controlado
Herramientas de Claude Code	`allowed-tools` en el frontmatter (límite duro)
Librerías de Python	imports fijos en `scrape.py` + `requirements.txt`

> Lo auditas **una sola vez**, no en cada sesión.

Estructura de la skill

```
scraper/
├── SKILL.md              # Manifiesto (lo que lee Claude)
├── README.md             # Portada para compartir
├── REFERENCE.md          # Recetas + troubleshooting
├── requirements.txt      # Dependencias fijadas
├── scripts/scrape.py     # Implementación (≈300 líneas)
└── tests/test_scrape.py
```

Arquitectura de `scrape.py`

- **`build_session`** — retry + backoff exponencial, respeta `Retry-After`
- **`RobotsGate`** — comprueba robots.txt por host (con cache)
- **`fetch`** — descarga con timeout; proxy ScraperAPI opcional
- **`extract_article`** — trafilatura: texto + metadatos
- **`extract_soup`** — BeautifulSoup + selector CSS
- **`load_done`** / **`append_record`** — reanudación + guardado incremental
- **`read_sitemap`** — descubrir URLs por sitemap.xml

Flujo de ejecución

```
URLs (--url / --urls / --urls-file / --sitemap)
   → dedup → filtrar ya-hechas (resume)
   → por cada URL:
        robots.txt OK? → fetch (retry) → extract → append JSONL
        (si falla: registra error y SIGUE; pausa con delay+jitter)
   → log final (ok / fail / robots-skip)
```

Un fallo **no aborta el lote**; un crash **no pierde** lo ya guardado.

Dos modos

`--mode article`	`--mode soup`
trafilatura	BeautifulSoup + CSS
texto limpio + metadatos	datos estructurados
artículos, noticias, blogs	precios, tablas, campos
corpus / NLP	selector específico del sitio

Robustez y ética integradas

- ✅ Retry + backoff (429/5xx) y `Retry-After`
- ✅ Respeto de **robots.txt** (desactivable con permiso)
- ✅ Guardado incremental + reanudación
- ✅ Cortesía entre requests (`--delay` + `--jitter`)
- ✅ Secretos vía `.env` (`os.getenv`), nunca hardcodeados
- ✅ `utf-8` , `pathlib` , `logging` ; salidas a `data/processed/`

Uso

```
# Artículo
python scripts/scrape.py --url URL --mode article --out out.jsonl

# Lote reanudable
python scripts/scrape.py --urls-file urls.txt --out noticias.jsonl

# Datos estructurados
python scripts/scrape.py --urls-file urls.txt --mode soup --selector ".precio"
```

Salida → pandas: `pd.read_json("out.jsonl", lines=True)`

Cómo compartirla

1. Copiar la carpeta a `~/.claude/skills/`
2. Versionar en un repo Git
3. Empaquetar como **plugin** (`/plugin install`)

¡Skill lista para producción!